

PharmaCast: Minimum Viable Product Report

Intelligent Pharmacy Inventory Management via Time-Series Forecasting and Asymmetric Cost Optimization

GROUP Project (Umais Siddiqui, David Greer, Inna Yedzinovich, Joao De Oliveira)

2026-03-15

Table of contents

1	Executive Summary	4
2	Step 1: Synthetic Data Generation	5
2.1	Rationale	5
2.2	Implementation (<code>data_generator.py</code> , 185 lines)	5
2.2.1	Drug Catalog	5
2.2.2	Demand Signal Composition	6
2.3	Output	6
3	Step 2: Data Preprocessing & Exploratory Data Analysis	7
3.1	Implementation (<code>data_preprocessing.py</code> , 192 lines)	7
3.1.1	Data Preparation	7
3.1.2	Train/Test Split	7
3.1.3	EDA Visualizations Generated	7
4	Step 3: Model Training with Facebook Prophet	8
4.1	Implementation (<code>model_training.py</code> , 306 lines)	8
4.1.1	Why Prophet?	8
4.1.2	Per-Drug Hyperparameter Tuning	8
4.1.3	Fallback: Holt-Winters Exponential Smoothing	8
4.1.4	Output	8
5	Step 4: Model Evaluation	9
5.1	Implementation (<code>model_evaluation.py</code> , 323 lines)	9
5.1.1	Primary Metric: Weighted MAPE (wMAPE)	9
5.1.2	Results	9
5.1.3	Interpretation	9

5.1.4	Cross-Validation	10
5.1.5	Evaluation Outputs	10
6	Step 5: Asymmetric Cost Functions	11
6.1	Implementation (<code>decision_engine.py</code> , 676 lines)	11
6.1.1	Wastage Cost C_w (Proposal §6.2)	11
6.1.2	Stockout Cost C_s (Proposal §6.3)	11
6.1.3	Total Asymmetric Loss L (Proposal §6.4)	11
6.1.4	Practical Example (Amoxicillin, 10-unit error)	11
7	Step 6: Cost-Optimal Order Quantity Q^*	12
7.1	Newsvendor Model (Proposal §7.2)	12
7.1.1	Demand Uncertainty Estimation	12
7.1.2	Expected Asymmetric Loss at Q^*	12
7.1.3	Inventory Simulation with Expiration	12
7.1.4	Reorder Recommendations Output	13
8	Step 7: Streamlit Dashboard	14
8.1	Implementation (<code>dashboard.py</code> , ~940 lines)	14
8.1.1	Page 1: Dashboard (KPI Overview)	14
8.1.2	Page 2: Forecasts	14
8.1.3	Page 3: Reorder Alerts	14
8.1.4	Page 4: Cost Analysis	14
8.1.5	Page 5: EDA Explorer	14
8.1.6	Page 6: Upload Data	15
8.1.7	Page 7: Admin (admin users only)	15
9	Step 8: Authentication System	16
9.1	Implementation (<code>auth_manager.py</code> , 258 lines; <code>user_db.py</code> , 183 lines)	16
9.1.1	Three Authentication Methods	16
9.1.2	User Database	16
9.1.3	Graceful Fallback	16
10	Step 9: Deployment & CI/CD	17
10.1	Docker Container (<code>Dockerfile</code>)	17
10.2	GitHub Actions CI/CD (<code>.github/workflows/deploy.yml</code>)	17
10.3	Google Cloud Storage Persistence	17
11	Step 10: Configuration System	18
11.1	YAML-Based Configuration (<code>data_adapter.py</code> , 308 lines)	18
11.2	Runtime Catalog Override	18
12	Summary: MVP Status vs. Proposal Requirements	19
12.1	Known Limitations	19

12.2 Repository Structure	20
-------------------------------------	----

1 Executive Summary

This report documents the Minimum Viable Product (MVP) implementation of **PharmaCast**, a machine learning-driven inventory management system designed for community pharmacies. The system addresses the “Optimization Paradox” — the fundamental tension between maintaining high stock levels for patient safety and minimizing waste from expired medications.

The MVP delivers:

- **Facebook Prophet** demand forecasting for 8 high-volatility medications with per-drug hyperparameter tuning
- **Asymmetric cost functions** ($\alpha = 10$) that penalize stockouts 10× more heavily than wastage
- **Cost-optimal order quantities** (Q^*) derived from the Newsvendor Model using forecast uncertainty
- A **production-deployed Streamlit dashboard** on Google Cloud Run with Google OAuth authentication
- **CI/CD pipeline** via GitHub Actions — push to **main** triggers automated build and deployment
- **Google Cloud Storage** persistence for data/model state across container restarts

Live Dashboard: <https://pharmacast-ze33vyu7q-uc.a.run.app>

2 Step 1: Synthetic Data Generation

2.1 Rationale

Real pharmacy sales data is protected under HIPAA and unavailable for academic research. We built a synthetic data engine that produces realistic daily prescription fill volumes calibrated against:

- CDC FluView seasonal curves (for winter/spring spike timing)
- CMS public prescription volume patterns
- US holiday calendars

2.2 Implementation (`data_generator.py`, 185 lines)

The data engine generates daily unit sales for 8 drugs across **788 days** (January 1, 2024 through February 26, 2026).

2.2.1 Drug Catalog

Drug	Category	Base Demand	Unit Cost	Shelf Life	Seasonal Profile
Amoxi-cillin 500mg	Antibiotic	18/day	\$0.45	365 days	Winter spike
Met-formin 500mg	Diabetes	30/day	\$0.12	730 days	Stable
Lisino-pril 10mg	Blood Pressure	25/day	\$0.08	730 days	Stable
Al-buterol Inhaler	Respiratory	8/day	\$25.00	365 days	Winter spike
Ceti-rizine 10mg	Allergy	12/day	\$0.15	730 days	Spring spike
Azithromycin 250mg	Antibiotic	10/day	\$1.20	365 days	Winter spike
Omepra-zole 20mg	GI	20/day	\$0.10	730 days	Holiday spike

Drug	Category	Base Demand	Unit Cost	Shelf Life	Seasonal Profile
Sertraline 50mg	Mental Health	15/day	\$0.18	730 days	Winter SAD

2.2.2 Demand Signal Composition

Each day’s demand for a drug is computed as:

$$D_t = \text{base} \times S_{\text{month}} \times W_{\text{dow}} \times (1 + g \cdot t) \times H_t + \epsilon_t$$

Where:

- S_{month} : Monthly seasonal multiplier (e.g., winter spike = 1.60 in January, 0.80 in July)
- W_{dow} : Day-of-week multiplier (Monday = 1.15, Sunday = 0.30)
- g : Trend growth rate (0.0003/day $\approx$ 11% annually)
- H_t : Holiday effect (0.30 on holidays, 1.25 day before)
- ϵ_t : Negative Binomial noise with drug-specific overdispersion (noise_scale 0.10–0.25)

2.3 Output

- **File:** data/pharmacy_sales.csv (~270 KB, ~6,300 records)
- **Columns:** date, drug_name, units_sold, category

3 Step 2: Data Preprocessing & Exploratory Data Analysis

3.1 Implementation (data_preprocessing.py, 192 lines)

3.1.1 Data Preparation

- Load and validate CSV with required columns (date, drug_name, units_sold, category)
- Convert to Prophet format: rename `date` → `ds`, `units_sold` → `y`
- Create US holiday DataFrame for Prophet with `lower_window=-1`, `upper_window=0`

3.1.2 Train/Test Split

- **Strategy:** Temporal split (no shuffling — critical for time-series)
- **Test set:** Last 60 days held out
- **Training set:** All preceding data (~728 days)

3.1.3 EDA Visualizations Generated

1. **Daily sales time series** — 2-column grid for all 8 drugs, showing seasonal patterns
2. **Monthly sales heatmap** — drug × month matrix revealing seasonal demand concentrations
3. **Day-of-week distribution** — boxplots confirming weekend demand drops (Saturday 55%, Sunday 30%)

4 Step 3: Model Training with Facebook Prophet

4.1 Implementation (model_training.py, 306 lines)

4.1.1 Why Prophet?

Prophet was selected based on the literature review (Rathipriya et al., 2023; Oliveira et al., 2023) for its:

- Automatic handling of yearly, weekly, and holiday seasonality
- Interpretable additive decomposition (trend + seasonality + holidays)
- Robustness to missing data and outliers
- Comparable accuracy to LSTM/XGBoost with less tuning overhead

4.1.2 Per-Drug Hyperparameter Tuning

Each drug gets individually tuned Prophet hyperparameters based on its demand characteristics:

Parameter	Stable Drugs	Seasonal Drugs
changepoint_prior_scale	0.01	0.10–0.15
seasonality_prior_scale	5.0	15.0–20.0
holidays_prior_scale	5.0	10.0–15.0

All drugs share:

- yearly_seasonality=True, weekly_seasonality=True, daily_seasonality=False
- Custom monthly seasonality added (period=30.5, Fourier order=5)
- interval_width=0.80 (80% confidence intervals)

4.1.3 Fallback: Holt-Winters Exponential Smoothing

A `HoltWintersWrapper` class provides a Prophet-compatible interface around `statsmodels.ExponentialSmoothing` for environments where Prophet's Stan backend is unavailable. Uses `trend="add"`, `seasonal="add"`, `seasonal_periods=7`, `damped_trend=True`.

4.1.4 Output

- **Models:** `models/{drug_name}_model.pkl` — serialized Prophet models (~89 KB each)
- **Forecasts:** 30-day predictions with `yhat`, `yhat_lower`, `yhat_upper`

5 Step 4: Model Evaluation

5.1 Implementation (model_evaluation.py, 323 lines)

5.1.1 Primary Metric: Weighted MAPE (wMAPE)

$$\text{wMAPE} = \frac{\sum |A_t - F_t|}{\sum |A_t|} \times 100$$

Volume-weighted MAPE is less sensitive to low-count weekend days where standard MAPE can inflate errors.

Project Target: wMAPE < 20%

5.1.2 Results

Drug	wMAPE	Target Met	80% CI Coverage
Metformin 500mg	<20%	Yes	–
Lisinopril 10mg	<20%	Yes	–
Omeprazole 20mg	<20%	Yes	–
Sertraline 50mg	<20%	Yes	–
Amoxicillin 500mg	>20%	No	–
Albuterol Inhaler	>20%	No	–
Cetirizine 10mg	>20%	No	–
Azithromycin 250mg	>20%	No	–

Average wMAPE: 23.6% (4/8 drugs pass) **Average 80% CI Coverage:** 63.5%

5.1.3 Interpretation

The results are consistent with published benchmarks:

- **Chronic/stable drugs** (Metformin, Lisinopril, Omeprazole, Sertraline): wMAPE < 20%, matching Rathipriya et al.'s 12–18% range
- **Seasonal/acute drugs** (Amoxicillin, Albuterol, Cetirizine, Azithromycin): wMAPE > 20%, consistent with their 25–35% range for seasonal drugs

Seasonal drugs have higher prediction error because their demand transitions (flu season onset, allergy season) are inherently harder to predict at exact timing.

5.1.4 Cross-Validation

Prophet's expanding-window cross-validation was used with:

- `initial="365 days", period="30 days", horizon="30 days"`
- Multiple cutoff points validate model stability across different forecast origins

5.1.5 Evaluation Outputs

- `outputs/evaluation_metrics.csv` — per-drug accuracy metrics
- `outputs/cross_validation_metrics.csv` — CV results
- Visualization: wMAPE bar chart with 20% target line

6 Step 5: Asymmetric Cost Functions

6.1 Implementation (decision_engine.py, 676 lines)

This is the core contribution of the project — translating forecast accuracy into economic decisions.

6.1.1 Wastage Cost C_w (Proposal §6.2)

$$C_w(t) = \max(F_t - A_t, 0) \times (c_{\text{unit}} \times p_{\text{exp}} + c_{\text{hold}})$$

Parameter	Value	Description
p_{exp} (short shelf)	0.05	Probability of expiration (365-day shelf life)
p_{exp} (long shelf)	0.02	Probability of expiration (730-day shelf life)
c_{hold}	\$0.03/unit/day	Daily holding cost

6.1.2 Stockout Cost C_s (Proposal §6.3)

$$C_s(t) = \max(A_t - F_t, 0) \times (\alpha \cdot c_{\text{unit}} + c_{\text{emerg}} + c_{\text{churn}})$$

Parameter	Value	Description
α	10	Asymmetric penalty multiplier
c_{emerg}	\$1.50/unit	Emergency reorder surcharge
c_{churn}	\$5.00/unit	Patient switching cost

6.1.3 Total Asymmetric Loss L (Proposal §6.4)

$$L(t) = C_w(t) + C_s(t)$$

6.1.4 Practical Example (Amoxicillin, 10-unit error)

- **Over-predict by 10:** $C_w = 10 \times (\$0.45 \times 0.05 + \$0.03) = \$0.53$
- **Under-predict by 10:** $C_s = 10 \times (10 \times \$0.45 + \$1.50 + \$5.00) = \$110$

The asymmetry is by design: stockouts carry **200×** **higher cost** per unit than wastage.

7 Step 6: Cost-Optimal Order Quantity Q^*

7.1 Newsvendor Model (Proposal §7.2)

The cost-optimal order quantity is derived from the classical Newsvendor framework:

$$\text{Critical Ratio} = \frac{C_s}{C_s + C_w}$$

$$Q^* = F^{-1}(\text{CR}, \mu = \text{forecast_mean}, \sigma = \text{std_est})$$

Where F^{-1} is the inverse CDF of the normal distribution.

7.1.1 Demand Uncertainty Estimation

Prophet's 80% confidence interval provides the uncertainty estimate:

$$\sigma_{\text{est}} = \frac{\text{yhat_upper} - \text{yhat_lower}}{2 \times z_{0.80}}$$

Where $z_{0.80} = 1.2816$.

7.1.2 Expected Asymmetric Loss at Q^*

$$E[L] = C_w \cdot \sigma \cdot (z\Phi(z) + \phi(z)) + C_s \cdot \sigma \cdot (-z\Phi(-z) + \phi(z))$$

Where $z = (Q^* - \mu)/\sigma$, Φ = normal CDF, ϕ = normal PDF.

7.1.3 Inventory Simulation with Expiration

The decision engine runs a 30-day inventory simulation with FIFO batch tracking:

1. Track batches as `(units, days_remaining)`
2. Age batches daily, mark expired units
3. Fulfill demand oldest-first (FIFO)
4. Compute service level and waste rate

$$\text{Service Level} = \frac{\text{Total Demand} - \text{Stockout Units}}{\text{Total Demand}} \times 100\%$$

$$\text{Waste Rate} = \frac{\text{Expired Units}}{\text{Starting Stock} + \text{Order}} \times 100\%$$

7.1.4 Reorder Recommendations Output

The engine generates per-drug recommendations with urgency levels:

Urgency	Condition	Action
CRITICAL	Days remaining < lead time	ORDER NOW
HIGH	Days remaining < lead time + safety stock	ORDER SOON
MEDIUM	Days remaining < 14	PLAN ORDER
LOW	Otherwise	MONITOR

8 Step 7: Streamlit Dashboard

8.1 Implementation (dashboard.py, ~940 lines)

The dashboard provides 7 interactive pages:

8.1.1 Page 1: Dashboard (KPI Overview)

- **Row 1:** Critical items, high priority, total order cost, drugs tracked
- **Row 2:** Total cost of ownership, wastage cost, understocking cost, avg service level
- **Row 3:** Asymmetric loss L_{total} , average critical ratio
- **Reorder recommendations table** with color-coded urgency
- **Model performance summary:** avg wMAPE and 80% CI coverage
- **Sales trend chart:** 90-day rolling daily total

8.1.2 Page 2: Forecasts

- Drug selector with 30-day Prophet forecast visualization
- Historical sales + forecast line with 80% confidence band
- Summary metrics: avg daily demand, peak day, total 30-day demand

8.1.3 Page 3: Reorder Alerts

- Per-drug urgency cards with detailed metrics
- Cost breakdown: order, wastage, understocking, total TCO
- 30-day stock depletion simulation chart with reorder point line

8.1.4 Page 4: Cost Analysis

- Portfolio cost summary (order, wastage, understocking, TCO)
- Stacked bar chart: TCO breakdown by drug
- Service level vs. waste rate scatter plot with target lines
- Full cost parameters reference (Proposal §6 formulas)

8.1.5 Page 5: EDA Explorer

- Multi-drug time series comparison (7-day rolling average)
- Day-of-week box plots
- Monthly demand trends (grouped bar chart)

8.1.6 Page 6: Upload Data

- CSV upload with validation (required columns, data types, date range)
- Optional inventory CSV for stock-on-hand
- Live model retraining with progress bar
- Automatic drug discovery from uploaded data

8.1.7 Page 7: Admin (admin users only)

- Authorized users table
- Grant Google access to new accounts
- Generate invite codes for email/password users
- Remove users (super admin protected)

9 Step 8: Authentication System

9.1 Implementation (`auth_manager.py`, 258 lines; `user_db.py`, 183 lines)

9.1.1 Three Authentication Methods

1. **Google OAuth 2.0** — Direct HTTP flow (no PKCE library dependency). User clicks “Sign in with Google”, is redirected to Google’s consent screen, and upon return the authorization code is exchanged for an ID token via a direct `urllib` POST to Google’s token endpoint.
2. **Email/Password** — SHA-256 hashed passwords with random salt. Requires prior account activation via invite code.
3. **Invite Code Registration** — Admin generates a short 8-character code, shares it with the user, who then sets their password on the login page.

9.1.2 User Database

JSON-backed storage at `data/users.json`, automatically synced to GCS.

- **Super admin:** `umaisabdullah@gmail.com` (hardcoded, non-removable)
- **Roles:** `admin` (full access + user management), `viewer` (dashboard access)

9.1.3 Graceful Fallback

If `GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET` environment variables are not set, the authentication system is bypassed entirely and the app runs in demo mode. This ensures the dashboard always works, even without OAuth configuration.

10 Step 9: Deployment & CI/CD

10.1 Docker Container (Dockerfile)

```
FROM python:3.12-slim
WORKDIR /app
RUN apt-get update && apt-get install -y build-essential curl
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN python run_pipeline.py      # Pre-train models during build
EXPOSE 8080
CMD streamlit run dashboard.py --server.port=${PORT} --server.headless=true
```

Key design decision: **models are trained during docker build**, so the container ships with pre-computed predictions and trained models. This ensures fast cold starts on Cloud Run.

10.2 GitHub Actions CI/CD (.github/workflows/deploy.yml)

Trigger: push to main (paths: FinalProjectData622/**)

Steps:

1. Checkout code
2. Authenticate to GCP (Workload Identity Federation)
3. Deploy to Cloud Run (source-based deploy)
 - Service: pharmacast
 - Region: us-central1
 - Memory: 2Gi, CPU: 2, Timeout: 300s
 - allow-unauthenticated (OAuth handled at app level)

10.3 Google Cloud Storage Persistence

cloud_storage.py syncs data/, models/, outputs/, and pharmacy_config.yaml to a GCS bucket (pharmacast-rxportal-data). On container startup, the dashboard's `_startup_gcs_sync()` function downloads any persisted state, ensuring uploaded pharmacy data and user accounts survive container restarts.

11 Step 10: Configuration System

11.1 YAML-Based Configuration (`data_adapter.py`, 308 lines)

The system uses a layered configuration approach:

1. **pharmacy_config.yaml** — pharmacist-editable, no code changes needed
2. **Environment variables** — deployment overrides (`PHARMACAST_DATA_DIR`, etc.)
3. **Hardcoded defaults** — fallback values in `config.py`

This design allows pharmacists to switch between synthetic and real data, change pharmacy names, and configure data sources without touching any Python code.

11.2 Runtime Catalog Override

When a pharmacist uploads custom CSV data via the Upload page, the system:

1. Auto-discovers drugs from the data using `detect_drugs()`
2. Builds a dynamic catalog with estimated base daily demand
3. Overrides the default drug catalog at runtime via `set_catalog_override()`
4. Trains new Prophet models for the discovered drugs
5. Generates fresh recommendations and cost analysis

12 Summary: MVP Status vs. Proposal Requirements

Proposal Requirement	Status	Notes
Prophet forecasting for 8 drugs	Implemented	Per-drug hyperparameter tuning
wMAPE < 20% target	Partial	4/8 pass; seasonal drugs above target (consistent with literature)
Asymmetric cost functions ($\alpha = 10$)	Implemented	Wastage C_w , stockout C_s , total L
Cost-optimal Q^* via Newsvendor	Implemented	Critical ratio quantile optimization
Inventory simulation with expiration	Implemented	FIFO batch tracking, service level, waste rate
Interactive Streamlit dashboard	Implemented	7 pages with Plotly interactive charts
Real pharmacy data support	Implemented	CSV upload with validation + live retraining
Cloud deployment	Implemented	Docker + Cloud Run + GCS + GitHub Actions CI/CD
Authentication	Implemented	Google OAuth + email/password + admin panel

12.1 Known Limitations

1. **Seasonal drug MAPE** — wMAPE > 20% for 4 seasonal drugs. Improvement paths: more training data (3+ years), higher Fourier orders for monthly seasonality, or hybrid models.

2. **Service level display** — Dashboard shows pre-reorder service level (~39%) which can be misleading. This is by design to show urgency, but could be augmented with post-order projected service level.
3. **Synthetic data only** — MVP uses synthetic data. Real pharmacy CSV upload is functional but not yet tested with production data.

12.2 Repository Structure

FinalProjectData622/

config.py	(254 lines)	Central configuration
data_generator.py	(185 lines)	Synthetic data engine
data_preprocessing.py	(192 lines)	Data cleaning & EDA
data_adapter.py	(308 lines)	YAML config & validation
model_training.py	(306 lines)	Prophet training
model_evaluation.py	(323 lines)	Accuracy metrics
decision_engine.py	(676 lines)	Cost functions & Q*
dashboard.py	(940 lines)	Streamlit dashboard
auth_manager.py	(258 lines)	Google OAuth & auth
user_db.py	(183 lines)	User database
cloud_storage.py	(116 lines)	GCS persistence
run_pipeline.py	(246 lines)	Pipeline orchestrator
Dockerfile		Container definition
requirements.txt		Python dependencies
pharmacy_config.yaml		Pharmacist config
.github/workflows/deploy.yml		CI/CD pipeline

Total: ~3,987 lines of Python across 12 modules.